

Names: David Fatheree, Preston May

```
`timescale 1ns / 1ps
```

```
module top(  
    input  i_rx,  
    input  clk,  
    output o_tx,  
    input  rst  
);
```

```
    // UART signals
```

```
    reg    tx_start;  
    wire   rx_valid, tx_done, tx_busy;  
    reg [127:0] tx_byte;  
    wire [255:0] rx_byte;
```

```
    // UART module
```

```
    uart_sm #(.BAUD(115200), .CLK_SPEED(12_000_000)) UART_LINK (  
        .clk(clk),  
        .reset(rst),  
        .i_rx(i_rx),  
        .o_tx(o_tx),  
        .tx_start(tx_start),  
        .tx_done(tx_done),  
        .tx_busy(tx_busy),  
        .rx_done_really(rx_valid),  
        .rx_words(rx_byte),  
        .tx_words(tx_byte)  
    );
```

```
    // FSM states
```

```
    localparam RST    = 3'b000,  
               WAIT   = 3'b001,  
               COMPUTE = 3'b010,  
               START_TX = 3'b011,  
               WAIT_TX = 3'b100,  
               WAIT2   = 3'b101;
```

```
    reg [2:0] state;
```

```

reg [127:0] byte;
reg [127:0] key_in;

wire [127:0] k0, k1;
wire [127:0] dummy[2:10];

key_generator u_keygen (
    .key_in(key_in),
    .k0(k0), .k1(k1),
    .k2(dummy[2]), .k3(dummy[3]), .k4(dummy[4]),
    .k5(dummy[5]), .k6(dummy[6]), .k7(dummy[7]),
    .k8(dummy[8]), .k9(dummy[9]), .k10(dummy[10])
);

wire [127:0] sub_bytes_out;
wire [127:0] shift_rows_out;
wire [127:0] mix_columns_out;

sub_bytes u_sub_bytes (
    .in(byte),
    .out(sub_bytes_out)
);

shift_rows u_shift_rows (
    .in(sub_bytes_out),
    .out(shift_rows_out)
);

mix_columns u_mix_columns (
    .in(shift_rows_out),
    .out(mix_columns_out)
);

// --- State Machine ---
always @(posedge clk) begin
    if (rst) begin
        state  <= RST;
        byte  <= 128'b0;
        key_in <= 128'b0;
        tx_byte <= 128'b0;
        tx_start <= 1'b0;
    end else begin
        case (state)
            RST: begin

```

```

        state <= WAIT;
    end

    WAIT: begin
        if (rx_valid) begin
            key_in <= rx_byte[255:128];
            byte <= rx_byte[127:0];
            state <= WAIT2;
        end
    end

    WAIT2: begin
        state <= COMPUTE;
    end

    COMPUTE: begin
        tx_byte <= mix_columns_out ^ k1; // AddRoundKey
        state <= START_TX;
    end

    START_TX: begin
        tx_start <= 1'b1;
        state <= WAIT_TX;
    end

    WAIT_TX: begin
        tx_start <= 1'b0;
        if (tx_done)
            state <= RST;
        end
    endcase
end
end
endmodule

```

```

`timescale 1ns / 1ps

module key_generator(
    input [127:0] key_in,
    output [127:0] k0,
    output [127:0] k1,
    output [127:0] k2,
    output [127:0] k3,
    output [127:0] k4,
    output [127:0] k5,
    output [127:0] k6,
    output [127:0] k7,
    output [127:0] k8,
    output [127:0] k9,
    output [127:0] k10
);

    // Round constants
    localparam [7:0] RC1 = 8'h01;
    localparam [7:0] RC2 = 8'h02;
    localparam [7:0] RC3 = 8'h04;
    localparam [7:0] RC4 = 8'h08;
    localparam [7:0] RC5 = 8'h10;
    localparam [7:0] RC6 = 8'h20;
    localparam [7:0] RC7 = 8'h40;
    localparam [7:0] RC8 = 8'h80;
    localparam [7:0] RC9 = 8'h1B;
    localparam [7:0] RC10 = 8'h36;

    // Round key 0 = original key
    assign k0 = key_in;

    // Sequential key expansion chain
    key_schedule r1 (.in(k0), .RC(RC1), .out(k1));
    key_schedule r2 (.in(k1), .RC(RC2), .out(k2));
    key_schedule r3 (.in(k2), .RC(RC3), .out(k3));
    key_schedule r4 (.in(k3), .RC(RC4), .out(k4));
    key_schedule r5 (.in(k4), .RC(RC5), .out(k5));

```

```
key_schedule r6 (.in(k5), .RC(RC6), .out(k6));
key_schedule r7 (.in(k6), .RC(RC7), .out(k7));
key_schedule r8 (.in(k7), .RC(RC8), .out(k8));
key_schedule r9 (.in(k8), .RC(RC9), .out(k9));
key_schedule r10 (.in(k9), .RC(RC10), .out(k10));
```

```
endmodule
```

```
module key_schedule(
    input [127:0] in,
    input [7:0] RC,
    output [127:0] out
);

    wire [31:0] w0 = in[127:96];
    wire [31:0] w1 = in[95:64];
    wire [31:0] w2 = in[63:32];
    wire [31:0] w3 = in[31:0];

    wire [31:0] w3_g;
    g_g_module(.in(w3), .RC(RC), .out(w3_g));

    wire [31:0] w0_out = w0 ^ w3_g;
    wire [31:0] w1_out = w1 ^ w0_out;
    wire [31:0] w2_out = w2 ^ w1_out;
    wire [31:0] w3_out = w3 ^ w2_out;

    assign out = {w0_out, w1_out, w2_out, w3_out};
```

```
endmodule
```

```
module g(
    input [31:0] in,
    input [7:0] RC,
    output [31:0] out
);

    wire [7:0] v [0:3];
    assign { v[0], v[1], v[2], v[3] } = in;

    wire [7:0] s0, s1, s2, s3;
```

```
s_box sb0 (v[1], s0);  
s_box sb1 (v[2], s1);  
s_box sb2 (v[3], s2);  
s_box sb3 (v[0], s3);
```

```
assign out = {s0 ^ RC, s1, s2, s3};
```

```
endmodule
```

```

`timescale 1ns/1ps

module tb_key_generator;

    // DUT inputs/outputs
    reg [127:0] key_in;
    wire [127:0] k0, k1, k2, k3, k4, k5, k6, k7, k8, k9, k10;

    // Instantiate Device Under Test
    key_generator dut (
        .key_in(key_in),
        .k0(k0), .k1(k1), .k2(k2), .k3(k3), .k4(k4),
        .k5(k5), .k6(k6), .k7(k7), .k8(k8), .k9(k9), .k10(k10)
    );

    // Expected round keys for AES-128 with key:
    // 2b7e151628aed2a6abf7158809cf4f3c
    localparam [127:0] EXP_K0 = 128'h2b7e1516_28aed2a6_abf71588_09cf4f3c;
    localparam [127:0] EXP_K1 = 128'ha0fafe17_88542cb1_23a33939_2a6c7605;
    localparam [127:0] EXP_K2 = 128'hf2c295f2_7a96b943_5935807a_7359f67f;
    localparam [127:0] EXP_K3 = 128'h3d80477d_4716fe3e_1e237e44_6d7a883b;
    localparam [127:0] EXP_K4 = 128'hef44a541_a8525b7f_b671253b_db0bad00;
    localparam [127:0] EXP_K5 = 128'hd4d1c6f8_7c839d87_caf2b8bc_11f915bc;
    localparam [127:0] EXP_K6 = 128'h6d88a37a_110b3efd_dbf98641_ca0093fd;
    localparam [127:0] EXP_K7 = 128'h4e54f70e_5f5fc9f3_84a64fb2_4ea6dc4f;
    localparam [127:0] EXP_K8 = 128'head27321_b58dbad2_312bf560_7f8d292f;
    localparam [127:0] EXP_K9 = 128'hac7766f3_19fadc21_28d12941_575c006e;
    localparam [127:0] EXP_K10 = 128'hd014f9a8_c9ee2589_e13f0cc8_b6630ca6;

    integer errors;

    // Simple compare task to reduce boilerplate
    task check_eq;
        input [127:0] got;
        input [127:0] exp;
        input [8*8:1] name; // small string literal like "k0"
    begin
        if (got != exp) begin

```

```

        errors = errors + 1;
        $display("[%0t] ERROR: %s mismatch", $time, name);
        $display("    Got: %032h", got);
        $display("    Exp: %032h", exp);
    end else begin
        $display("[%0t] OK  : %s = %032h", $time, name, got);
    end
end
endtask

initial begin
    errors = 0;

    // Drive the canonical AES-128 key
    key_in = 128'h2b7e1516_28aed2a6_abf71588_09cf4f3c;

    // Wait a delta cycle for combinational propagation
    #1;

    // Check all round keys
    check_eq(k0 , EXP_K0 , "k0" );
    check_eq(k1 , EXP_K1 , "k1" );
    check_eq(k2 , EXP_K2 , "k2" );
    check_eq(k3 , EXP_K3 , "k3" );
    check_eq(k4 , EXP_K4 , "k4" );
    check_eq(k5 , EXP_K5 , "k5" );
    check_eq(k6 , EXP_K6 , "k6" );
    check_eq(k7 , EXP_K7 , "k7" );
    check_eq(k8 , EXP_K8 , "k8" );
    check_eq(k9 , EXP_K9 , "k9" );
    check_eq(k10, EXP_K10, "k10");

    // Final report
    if (errors == 0) begin
        $display("-----");
        $display("ALL TESTS PASSED: key_generator matches FIPS-197 (AES-128).");
        $display("-----");
    end else begin
        $display("-----");
        $display("TESTS FAILED: %0d mismatches detected.", errors);
        $display("-----");
    end

    // End simulation

```



```

        #5;
        $finish;
    end

endmodule

`timescale 1ns/1ps

module tp_module_top_replica (
    input      clk,
    input      rst,
    input      start,
    input  [127:0] key_in,
    input  [127:0] byte_in,
    output reg [127:0] tx_word,
    output reg      tx_word_valid
);

    // Key expansion
    wire [127:0] k0, k1;
    wire [127:0] dummy [2:10];

    key_generator u_keygen (
        .key_in(key_in),
        .k0(k0), .k1(k1),
        .k2(dummy[2]), .k3(dummy[3]), .k4(dummy[4]),
        .k5(dummy[5]), .k6(dummy[6]), .k7(dummy[7]),
        .k8(dummy[8]), .k9(dummy[9]), .k10(dummy[10])
    );

    // AES first-round data path (as in top.v)
    wire [127:0] sub_bytes_out;
    wire [127:0] shift_rows_out;
    wire [127:0] mix_columns_out;

    sub_bytes u_sub_bytes (.in(byte_in), .out(sub_bytes_out));
    shift_rows u_shift_rows (.in(sub_bytes_out), .out(shift_rows_out));
    mix_columns u_mix_cols (.in(shift_rows_out), .out(mix_columns_out));

    // Latch result on 'start' to mirror COMPUTE -> START_TX handoff point
    always @(posedge clk) begin
        if (rst) begin
            tx_word      <= 128'b0;
            tx_word_valid <= 1'b0;
        end else begin

```

```

        tx_word_valid <= 1'b0;
    if (start) begin
        tx_word    <= mix_columns_out ^ k1;
        tx_word_valid <= 1'b1;
    end
end
end

endmodule

// -----
// Testbench: tb_top_sans_uart
// Drives key and plaintext directly.
// Prints the 128-bit tx_word produced by the replicated pipeline.
// -----
module tb_top_all_modules;

    // Clock/reset
    reg clk;
    reg rst;

    // Stimulus
    reg start;
    reg [127:0] key_in;
    reg [127:0] byte_in;

    // Outputs from DUT
    wire [127:0] tx_word;
    wire tx_word_valid;

    tp_module_top_replica dut (
        .clk(clk),
        .rst(rst),
        .start(start),
        .key_in(key_in),
        .byte_in(byte_in),
        .tx_word(tx_word),
        .tx_word_valid(tx_word_valid)
    );

    localparam integer CLK_HZ    = 12_000_000;
    localparam integer CLK_PERIOD = 1_000_000_000 / CLK_HZ; // ns

    initial begin

```

```

    clk = 1'b0;
    forever #(CLK_PERIOD/2) clk = ~clk;
end

initial begin
    #10_000_000;
    $display("[%0t] TIMEOUT: ending simulation via watchdog.", $time);
    $finish;
end

// Pretty-print helper
task print128;
    input [127:0] v;
    integer i;
    reg [7:0] b [0:15];
begin
    {b[0], b[1], b[2], b[3], b[4], b[5], b[6], b[7],
     b[8], b[9], b[10], b[11], b[12], b[13], b[14], b[15]} = v;
    $write("0x");
    for (i = 0; i < 16; i = i + 1) $write("%02x", b[i]);
end
endtask

initial begin
    // Reset
    start = 1'b0;
    key_in = 128'b0;
    byte_in = 128'b0;

    rst = 1'b1;
    repeat (8) @(posedge clk);
    rst = 1'b0;
    repeat (4) @(posedge clk);

    // ----- Test Case 1 -----
    key_in = 128'h2b7e1516_28aed2a6_abf71588_09cf4f3c;
    byte_in = 128'h3243f6a8_885a308d_313198a2_e0370734;

    // Pulse 'start' for 1 cycle
    @(posedge clk); start <= 1'b1;
    @(posedge clk); start <= 1'b0;

    wait (tx_word_valid === 1'b1);
    #1;

```

```

$write("[%0t] TX_WORD (Test 1) = ", $time);
print128(tx_word);
$write("\n");

// ----- Test Case 2 -----
key_in = 128'h00000000_00000000_00000000_00000000;
byte_in = 128'h00000000_00000000_00000000_00000000;

@(posedge clk); start <= 1'b1;
@(posedge clk); start <= 1'b0;

wait (tx_word_valid === 1'b1);
#1;

$write("[%0t] TX_WORD (Test 2) = ", $time);
print128(tx_word);
$write("\n");

// Grace delay then finish
#(10*CLK_PERIOD);
$display("[%0t] TB done. Calling $finish.", $time);
$finish;
end

endmodule

```

```
import sys, time, serial, random
```

```
SERIAL_PORT = "COM3"
```

```
BAUD_RATE = 115200
```

```
NUM_TESTS = 20
```

```
SBOX = [
```

```
0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76,  
0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0,  
0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15,  
0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75,  
0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84,  
0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf,  
0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8,  
0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2,  
0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73,  
0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb,  
0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79,  
0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08,  
0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a,  
0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e,  
0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf,  
0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16  
]
```

```
def sub_bytes(state_bytes):
```

```
    return bytes(SBOX[b] for b in state_bytes)
```

```
def shift_rows(state_bytes):
```

```
    s = list(state_bytes)
```

```
    m = [
```

```
        [s[0], s[4], s[8], s[12]], # row 0
```

```
        [s[1], s[5], s[9], s[13]], # row 1
```

```
        [s[2], s[6], s[10], s[14]], # row 2
```

```
        [s[3], s[7], s[11], s[15]], # row 3
```

```
    ]
```

```
    out = []
```

```
    for i, row in enumerate(m):
```

```
        out.extend(row[i:] + row[:i])
```

```

return bytes(out)

def mul(a, b):
    res = 0
    for _ in range(8):
        if b & 1:
            res ^= a
        high = a & 0x80
        a = (a << 1) & 0xFF
        if high:
            a ^= 0x1B
        b >>= 1
    return res

def mix_single_column(col):
    a = list(col)
    return bytes([
        mul(2,a[0]) ^ mul(3,a[1]) ^ a[2] ^ a[3],
        a[0] ^ mul(2,a[1]) ^ mul(3,a[2]) ^ a[3],
        a[0] ^ a[1] ^ mul(2,a[2]) ^ mul(3,a[3]),
        mul(3,a[0]) ^ a[1] ^ a[2] ^ mul(2,a[3])
    ])

def mix_columns(state):
    s = [list(state[i::4]) for i in range(4)]
    mixed = [mix_single_column(col) for col in s]
    out = bytearray(16)
    for c in range(4):
        for r in range(4):
            out[r + 4*c] = mixed[c][r]
    return bytes(out)

def sub_word(w):
    return ((SBOX[(w>>24)&0xFF]<<24)|
            (SBOX[(w>>16)&0xFF]<<16)|
            (SBOX[(w>>8)&0xFF]<<8)|
            SBOX[w&0xFF])

def rot_word(w):
    return ((w<<8)&0xFFFFFFFF)|(w>>24)

def next_round_key(key_bytes, RC=0x01):
    w=[int.from_bytes(key_bytes[i:i+4],'big') for i in range(0,16,4)]
    temp=sub_word(rot_word(w[3]))^(RC<<24)

```

```

w4=w[0]^temp
w5=w[1]^w4
w6=w[2]^w5
w7=w[3]^w6
return b''.join(x.to_bytes(4,'big') for x in [w4,w5,w6,w7])

def one_round_encrypt(state,key):
    k1=next_round_key(key)
    s=sub_bytes(state)
    s=shift_rows(s)
    s=mix_columns(s)
    return bytes(a^b for a,b in zip(s,k1))

def send_and_receive(port,baud,data):
    with serial.Serial(port,baud,timeout=1.0) as ser:
        ser.reset_input_buffer()
        ser.reset_output_buffer()
        time.sleep(0.02)
        ser.write(data)
        ser.flush()
        return ser.read(16)

def main():
    print(f"Testing one AES round via UART on {SERIAL_PORT} @ {BAUD_RATE}")
    total = 0
    passed = 0
    for i in range(NUM_TESTS):
        key=bytes(random.getrandbits(8) for _ in range(16))
        plaintext=bytes(random.getrandbits(8) for _ in range(16))

        expected=one_round_encrypt(plaintext,key)

        rx=send_and_receive(SERIAL_PORT,BAUD_RATE,key+plaintext)

        print(f"\nTest {i+1:02d}")
        print(f" Key      : {key.hex()}")
        print(f" Plaintext  : {plaintext.hex()}")
        print(f" Expected   : {expected.hex()}")
        ok = (len(rx) == 16 and rx == expected)
        if len(rx) != 16:
            print(f" Received   : (incomplete, {len(rx)} bytes) {rx.hex()}")
        else:
            print(f" Received : {rx.hex()}")
        print(" → PASS" if ok else " → FAIL")

```

```
    total += 1
    passed += ok
    print(f"\n{passed}/{NUM_TESTS} tests passed")

if __name__ == "__main__":
    main()
```


[1000] OK : k0 = 2b7e151628aed2a6abf7158809cf4f3c
[1000] OK : k1 = a0fafe1788542cb123a339392a6c7605
[1000] OK : k2 = f2c295f27a96b9435935807a7359f67f
[1000] OK : k3 = 3d80477d4716fe3e1e237e446d7a883b
[1000] OK : k4 = ef44a541a8525b7fb671253bdb0bad00
[1000] OK : k5 = d4d1c6f87c839d87caf2b8bc11f915bc
[1000] OK : k6 = 6d88a37a110b3efddb98641ca0093fd
[1000] OK : k7 = 4e54f70e5f5fc9f384a64fb24ea6dc4f
[1000] OK : k8 = ead27321b58dbad2312bf5607f8d292f
[1000] OK : k9 = ac7766f319fadc2128d12941575c006e
[1000] OK : k10 = d014f9a8c9ee2589e13f0cc8b6630ca6

ALL TESTS PASSED: key_generator matches FIPS-197 (AES-128).

[1108000] TX_WORD (Test 1) = 0x616cc7ba4e93e37b1cd00709e38fcb57
[1272000] TX_WORD (Test 2) = 0x01000000010000000100000001000000
[2102000] TB done. Calling \$finish.

